QueueCTL — Backend Developer Internship Assignment

Tech Stack: Your choice — Python / Go / Node.js / Java
Submission: Public GitHub repository + README + live review session

---

🎯 Objective

Build a CLI-based background job queue system called `queuectl`.

The system manages background jobs with worker processes, retries failures with exponential backoff, and maintains a Dead Letter Queue (DLQ) for permanently failed jobs.

> **Read this first:** Your submission is evaluated in two parts — the code **and a 30-minute live review** where you explain and modify your own system. You may use any tools you like to build it (including AI assistants), but you must be able to explain, defend, and change **every line** on a screen share. A submission whose author cannot explain it is rejected regardless of how well it works.

🧩 Problem Overview

Implement a minimal, production-grade job queue that supports:

- Enqueuing and managing background jobs
- Running multiple worker processes **in parallel, from separate terminals**
- Automatic retries with exponential backoff

- A Dead Letter Queue after retries are exhausted
- Persistent job storage across restarts **and crashes**
- All operations through a CLI

📦 Job Specification

Each job must contain at least:

```
{
  "id": "unique-job-id",
  "command": "echo 'Hello World'",
  "state": "pending",
  "attempts": 0,
  "max_retries": 3,
  "created_at": "2025-11-04T10:30:00Z",
  "updated_at": "2025-11-04T10:30:00Z"
}
```

🔄 Job Lifecycle

State	Description
`pending`	Waiting to be picked up by a worker
`processing`	Currently being executed
`completed`	Successfully executed
`failed`	Failed, but retryable (waiting for its backoff delay)
`dead`	Permanently failed (moved to DLQ)

Crash rule: a job must never be stuck in `processing` forever. If the worker running it dies (including `SIGKILL` — no cleanup handler will run), the system must detect this and recover the job so it can run again. With your default settings, worst-case recovery must be **under 60 seconds** — our automated tests enforce this. Document your recovery mechanism and its trade-offs in `DECISIONS.md`.

💻 CLI Commands

Category	Command Example	Description
Enqueue	`queuectl enqueue '{"id":"job1","command":"sleep 2"}'`	Add a new job
Workers	`queuectl worker start --count 3`	Start workers **in the foreground** (blocks until stopped)
	`queuectl worker stop`	Gracefully stop all running workers **from another terminal**
Status	`queuectl status`	Summary of all job states & active workers
List	`queuectl list --state pending [--json]`	List jobs by state
DLQ	`queuectl dlq list` / `queuectl dlq retry job1`	View or retry DLQ jobs
Config	`queuectl config set max-retries 3`	Manage configuration

Interface contract (required — our automated test suite depends on it):

1. `worker start` runs in the **foreground**. `SIGTERM` / `SIGINT` (Ctrl+C) triggers **graceful shutdown**: finish the current job, then exit. `SIGKILL` simulates a crash — your system must survive it (see crash rule).
2. `queuectl list --state <state> --json` prints a JSON **array** of job objects to stdout (and nothing else on stdout).
3. `worker stop` must work from a *different terminal* than the one running the workers. How your CLI discovers and signals live workers (PID files, control socket, DB rows, ...) is a design decision — document what you chose and what you rejected in `DECISIONS.md`.

⚙️ System Requirements

1. **Job Execution**
    - Workers execute the job's `command` via the shell; the exit code determines success (0) or failure (non-zero, including command-not-

found).

2. **Retry & Backoff**
   - Failed jobs retry automatically after a delay of `delay = base ^ attempts` seconds, where `attempts` is the number of completed attempts (so with `base = 2`: first retry after 2s, then 4s, 8s).
   - Default `base` is 2, configurable via `config set backoff-base`.
   - After `max_retries` failed attempts, the job moves to the DLQ (`dead`).
   - `dlq retry <id>` re-enqueues a dead job. Decide whether it resets `attempts` and justify the choice in `DECISIONS.md`.
3. **Persistence**
   - All job data survives process restarts. File-based JSON, SQLite, or anything you can justify — but your locking story (below) must actually hold for what you pick.
4. **Worker Management & Concurrency**
   - Multiple workers run in parallel — including workers started from **separate terminal sessions** (separate OS processes, not just threads).
   - A job must never be executed by two workers at once. In `DECISIONS.md`, point to the **exact line(s)** that make claiming a job atomic and explain *why* the mechanism is atomic across processes.
   - Graceful shutdown: on `worker stop` or Ctrl+C, finish the in-flight job.
5. **Configuration**
   - Retry count and backoff base configurable via CLI, persisted.
   - Document whether config changes affect already-enqueued jobs.

🧪 Automated Testing (live, during the interview)

During your interview session, we will run **our own automated test script** against your submission on a screen share — you will not see the script beforehand. It drives your real CLI (via the interface contract above) and exercises at least these scenarios:

1. A basic job completes.
2. A failing job retries with backoff and lands in the DLQ.
3. Many jobs across multiple workers — every job runs **exactly once**.
4. Workers are `SIGKILL`ed mid-job; after restart, every job still completes and nothing is stuck in `processing`.
5. Jobs survive a full restart.

Failing scenarios 1–3 in the live run ends the interview. If a scenario fails, you'll be asked to diagnose it on the spot — how you debug your own system under pressure is part of the evaluation. Build and test against these scenario descriptions yourself; the script only automates them.

The script depends on the interface contract exactly as written (foreground workers, `--json` output, signal semantics). Deviating from the contract will fail the run regardless of how correct your logic is.

📋 Deliverables

- ✅ Working `queuectl` CLI (all commands above, matching the interface contract)
- ✅ Handles test scenarios 1–5 above (verify them yourself before submitting)
- ✅ Persistent storage, retry/backoff, DLQ, crash recovery
- ✅ `README.md` — setup, usage examples, architecture overview, testing
- ✅ `DECISIONS.md` — see below
- ✅ **Incremental git history** — commit as you work. A history that jumps from empty to finished in one or two commits will be questioned in review. We read the history as part of grading.
- ✅ Short CLI demo recording (link in README)

DECISIONS.md (required)

Answer these five questions specifically — vague answers score zero:

1. Which exact line(s) prevent two workers from claiming the same job, and why is that operation atomic **across separate OS processes**?
2. A worker is `SIGKILL`ed halfway through a job. Walk through, step by step, what state the job is in and how it eventually runs again. What is the worst-case delay before recovery?
3. Does `dlq retry` reset `attempts`? Why is that the right call?
4. What designs did you consider and **reject** for `worker stop` (cross-process signaling), and why?
5. If priorities were added tomorrow (high-priority jobs jump the queue), which parts of your design survive unchanged and which break?

🗣️ Live Review (30 min, mandatory)

Shortlisted candidates do a screen-share session:

- **Automated test run (~10 min):** we run our test script against your code on your machine. Failures become live debugging — talk us through your diagnosis.
- **Defense (~10 min):** questions about your design and edge cases — in the spirit of the `DECISIONS.md` questions, but going deeper.
- **Live change (~10 min):** you implement one small requirement change to your own code, live (e.g., a new command or a behavior tweak). We're watching how you navigate your own codebase, not whether you finish.

Have your environment ready: your repo cloned and runnable, `bash` and `python3` available.

You may also receive **one requirement change by email after submitting**, with 48 hours to implement it. Design for change.

📊 Evaluation Criteria

Criteria	Weight	Description
Automated test run (live)	Gate	Scenarios 1–3 fail → interview ends

Criteria	Weight	Description
Functionality	20%	Full command surface, DLQ, config
Robustness	20%	Crash recovery, concurrency safety, edge cases
Live review	30%	Can you explain and modify your own system?
Code quality	15%	Structure, readability, idiomatic use of your stack
DECISIONS.md + README	15%	Specific, honest reasoning; real trade-offs

🌟 Bonus (optional)

Job timeouts · priority queues · scheduled jobs (`run_at`) · job output logging · metrics · minimal web dashboard

⚠️ Disqualification

- Fails scenarios 1–3 in the live automated test run
- Duplicate job execution or jobs lost on restart
- Jobs permanently stuck in `processing` after a worker crash
- Cannot explain your own code in the live review
- Missing `DECISIONS.md` or generic/evasive answers in it

🧾 Submission

1. Push to a **public GitHub repository** (with your real incremental history).
2. Include `README.md`, `DECISIONS.md`, and the demo recording link.
3. Share the repository link for review.